

IOC Checker

Automated Threat Intelligence Tool

A Python command-line tool that automates threat intelligence lookups across AbuseIPDB, VirusTotal, and Shodan — then generates a weighted risk score to help analysts decide whether to block or ignore a suspicious IP address.

The Problem

Analysts had to visit AbuseIPDB, VirusTotal, and Shodan separately for every suspicious IP — repetitive and time-consuming.

The Solution

One command queries all three platforms, calculates a risk score, and delivers a clear verdict in seconds.

How It Works

Step	Action	Detail
1	Run the script	<code>python ioc_checker.py</code>
2	Enter an IP address	Enter IP: 8.8.8.8
3	Queries 3 threat platforms	AbuseIPDB · VirusTotal · Shodan
4	Calculates risk score	$(Abuse \times 0.5) + (VT \times 10) + (Ports \times 2)$
5	Displays verdict + saves JSON	LOW / MEDIUM / HIGH / EXTREME

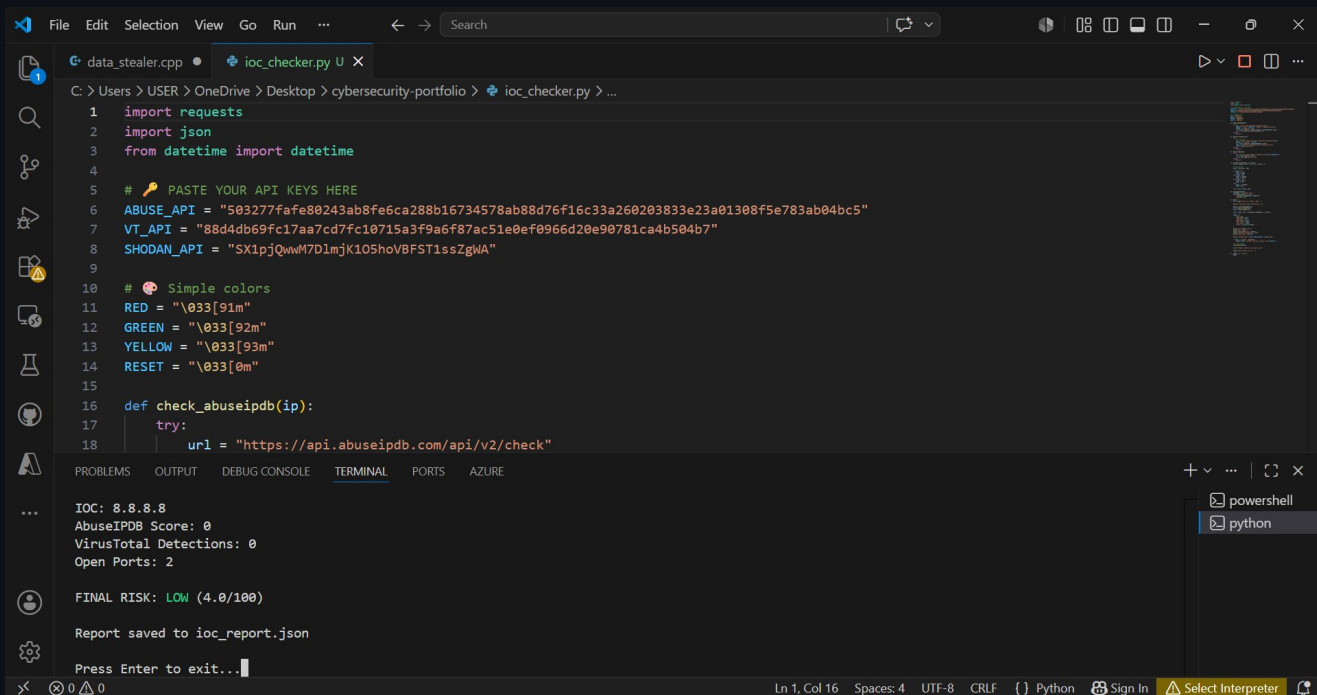
Risk Level Thresholds

Score	Level	Recommended Action
0 – 24	LOW	Monitor — likely benign
25 – 49	MEDIUM	Investigate further
50 – 74	HIGH	Escalate to senior analyst
75 – 100	EXTREME	Block immediately

Test Results

Test 1 — Clean IP: 8.8.8.8 (Google DNS)

Google's public DNS resolver is a globally trusted address. The tool correctly identified it as LOW risk — zero abuse reports, zero malware detections, two open (expected) ports.



```
1 import requests
2 import json
3 from datetime import datetime
4
5 # PASTE YOUR API KEYS HERE
6 ABUSE_API = "503277fafe80243ab8fe6ca288b16734578ab88d76f16c33a260203833e23a01308f5e783ab04bc5"
7 VT_API = "88d4db69fc17aa7cd7fc10715a3f9a6f87ac51e0ef0966d20e90781ca4b504b7"
8 SHODAN_API = "SX1pjQwwM7DlmjK10ShoVBFST1ssZgWA"
9
10 # Simple colors
11 RED = "\033[91m"
12 GREEN = "\033[92m"
13 YELLOW = "\033[93m"
14 RESET = "\033[0m"
15
16 def check_abuseipdb(ip):
17     try:
18         url = "https://api.abuseipdb.com/api/v2/check"
```

```
IOC: 8.8.8.8
AbuseIPDB Score: 0
VirusTotal Detections: 0
Open Ports: 2

FINAL RISK: LOW (4.0/100)

Report saved to ioc_report.json

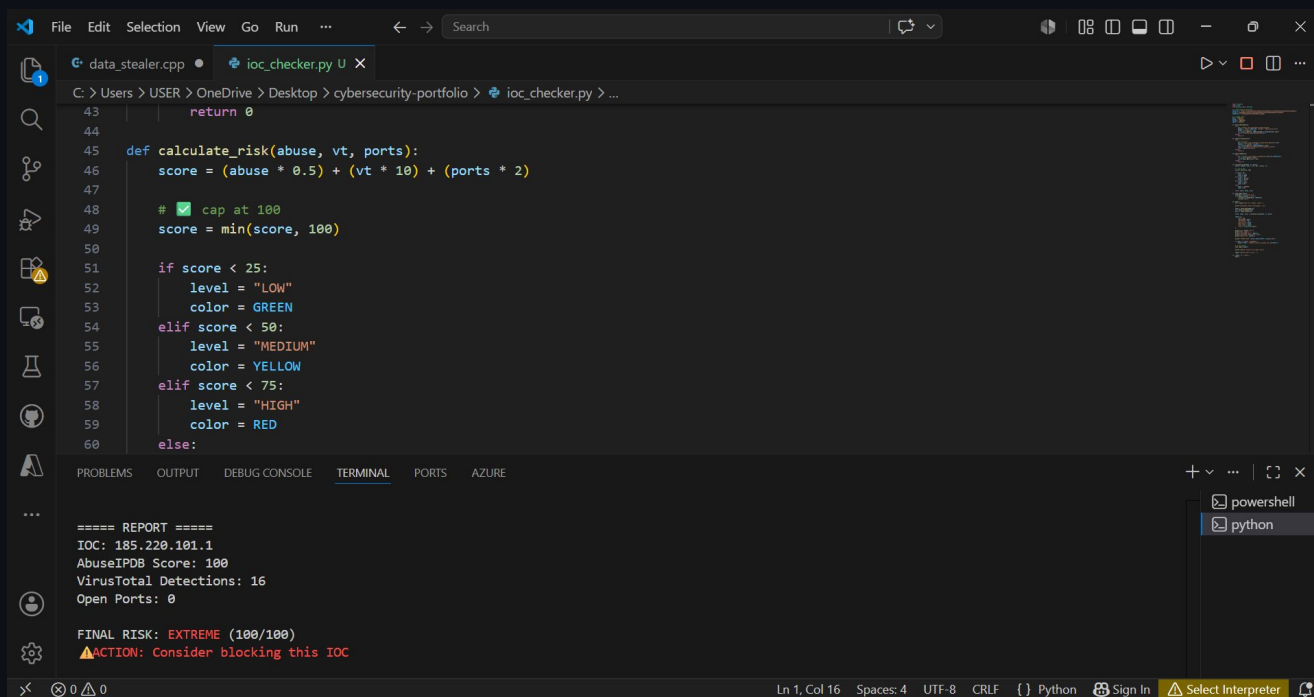
Press Enter to exit...
```

Figure 1 · Terminal output — FINAL RISK: LOW (4.0/100)

Metric	Result
AbuseIPDB Score	0 — No abuse reports
VirusTotal Detections	0 — Clean
Open Ports (Shodan)	2 — Normal for a DNS server
Final Risk Score	4.0 / 100 — LOW ✓

Test 2 — Malicious IP: 185.220.101.1

A known Tor exit node flagged across multiple threat feeds. The tool instantly rated it EXTREME with a perfect score of 100/100, and output an action recommendation to block the IP.



```
43     return 0
44
45 def calculate_risk(abuse, vt, ports):
46     score = (abuse * 0.5) + (vt * 10) + (ports * 2)
47
48     # ✅ cap at 100
49     score = min(score, 100)
50
51     if score < 25:
52         level = "LOW"
53         color = GREEN
54     elif score < 50:
55         level = "MEDIUM"
56         color = YELLOW
57     elif score < 75:
58         level = "HIGH"
59         color = RED
60     else:
61         level = "EXTREME"
62         color = RED
63
64     return level, color
65
66 if __name__ == '__main__':
67     ioc = '185.220.101.1'
68     abuse = 100
69     vt = 16
70     ports = 0
71
72     level, color = calculate_risk(abuse, vt, ports)
73
74     print(f"IOC: {ioc} | Risk Level: {level} | Color: {color}")
75
76     # Final Risk Calculation
77     final_risk = f"{abuse} / {vt} — {level}"
78
79     print(f"Final Risk: {final_risk}")
80
81     # Action Recommendation
82     if level == "EXTREME":
83         action = "Consider blocking this IOC"
84     else:
85         action = "Monitor closely"
86
87     print(f"Action: {action}")
88
89     # Print Report
90     print("\n===== REPORT =====")
91     print(f"IOC: {ioc}")
92     print(f"AbuseIPDB Score: {abuse}")
93     print(f"VirusTotal Detections: {vt}")
94     print(f"Open Ports: {ports}")
95
96     print(f"Final Risk: {final_risk}")
97     print(f"Action: {action}")
98
99     print("\n===== END REPORT =====")
```

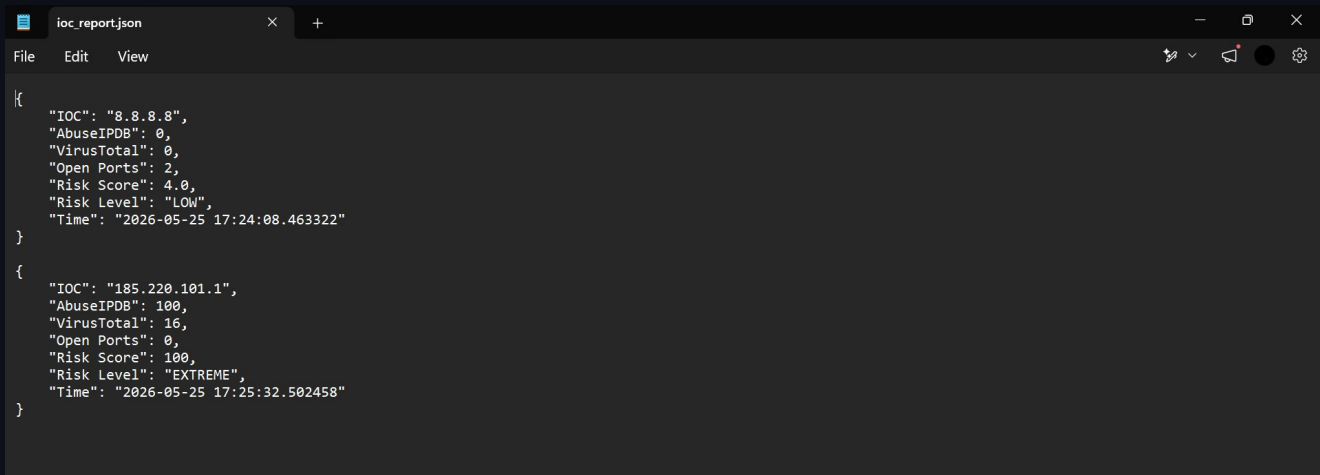
=====
IOC: 185.220.101.1
AbuseIPDB Score: 100
VirusTotal Detections: 16
Open Ports: 0
Final Risk: EXTREME (100/100)
ACTION: Consider blocking this IOC

Figure 2 · Terminal output — FINAL RISK: EXTREME (100/100) · ACTION: Consider blocking this IOC

Metric	Result
AbuseIPDB Score	100 — Maximum abuse confidence
VirusTotal Detections	16 — Multiple AV engines flagged
Open Ports (Shodan)	0 — Stealthy / no exposed services
Final Risk Score	100 / 100 — EXTREME ■

JSON Report File

After every scan the tool appends a structured JSON record to `ioc_report.json`. This creates a persistent audit trail that can be reviewed, shared with a team, or ingested by a SIEM.



```
{
  "IOC": "8.8.8.8",
  "AbuseIPDB": 0,
  "VirusTotal": 0,
  "Open Ports": 2,
  "Risk Score": 4.0,
  "Risk Level": "LOW",
  "Time": "2026-05-25 17:24:08.463322"
}

{
  "IOC": "185.220.101.1",
  "AbuseIPDB": 100,
  "VirusTotal": 16,
  "Open Ports": 0,
  "Risk Score": 100,
  "Risk Level": "EXTREME",
  "Time": "2026-05-25 17:25:32.502458"
}
```

Figure 3 · `ioc_report.json` — both test results saved with timestamp, scores, and risk level

Key Skills Demonstrated

- Threat Intelligence Integration**
Queried AbuseIPDB, VirusTotal, and Shodan in real time — the same platforms used by professional SOC teams worldwide.
- Automation / SOAR Mindset**
Replaced three manual browser lookups with a single command, cutting analysis time and reducing human error.
- Decision Logic & Scoring**
Raw API data is weighted and combined into a single interpretable risk score — the tool doesn't just collect data, it reasons about it.
- Structured Reporting**
JSON output enables logging, trend analysis, and easy integration with downstream security tooling or dashboards.

Leonard N